

Plain-language explanation of three recent articles for Kasi's thesis

Student: Kasireddy Vadicharla (25104047)

Programme: MSc Cloud Computing, National College of Ireland (NCI)

Thesis focus: Source-free log anomaly detection for AWS serverless applications (CloudWatch logs with injected faults), with Zhao et al. (2025) ELFA-Log as the main transfer-learning comparator.

This short note explains, in everyday language, the three recent studies that sit behind the project's gap statement. It is written so a professor can see what each paper did, what it found, how it is organised, and what space it leaves for Kasi's work. Jargon is defined the first time it appears. Numbers below are taken only from the paper texts we downloaded and checked.

Why these three articles matter

Kasi's project asks a practical question: can we detect faults in AWS serverless (Lambda-style) CloudWatch logs when we have **no labelled source system** to copy from? "Labelled logs" means each log sequence (or message) has already been marked as normal or anomalous by a human or by a known fault campaign. Many published detectors need those labels, at least on a **source** system, and then try to **transfer** that knowledge to a new **target** system. That idea is called **transfer learning**.

The three articles below are the recent anchors of the gap:

1. **Zhao et al. (2025)** give a strong **transfer** comparator (ELFA-Log), but they work on public **benchmark corpora** (HDFS, BGL, OpenStack) and still rely on a labelled source.
2. **Ali et al. (2025)** study what happens when you **drop** heavy reliance on labels (semi-supervised learning) and they also measure **time cost**, but again on those same kinds of benchmark corpora—not on serverless CloudWatch telemetry.
3. **Nguyen et al. (2025)** focus on **serverless** telemetry and silent failures, but the paper is a **vision** paper: it sets out challenges and a research agenda; it does **not** evaluate a finished anomaly detector the way a methods paper would.

Kasi's planned study aims to sit in the hole left between them: **no labelled source, serverless CloudWatch logs, injected-fault ground truth**, and a **transfer comparator** (ELFA-Log) so the accuracy cost of going source-free can be measured fairly.

Article 1 — Zhao et al. (2025): ELFA-Log (baseline / comparator)

Full citation: Zhao, X., Guo, K., Huang, M., Qiu, S. and Lu, L. (2025) ‘ELFA-Log: Cross-System Log Anomaly Detection via Enhanced Pseudo-Labeling and Feature Alignment’, *Computers*, 14(7), 272. <https://doi.org/10.3390/computers14070272>

Source of PDF used here: MDPI open-access copy already held in baseline_papers/Zhao_et_al_2025_ELFA (already on the Mac thesis folder) (verified title and authors on page 1; 21 pages).

Purpose (what problem it tackles)

Most log anomaly detectors need large volumes of labelled training data. That is hard when a **new** system arrives with few or no labels. Cross-system log anomaly detection (CSLAD) tries to reuse a model trained on a **source** system to help a **target** system. Zhao et al. argue that older transfer methods still struggle when the source and target log **distributions** differ (different formats, event types, and system behaviour). Their answer is **ELFA-Log**: a transfer-learning method that (a) creates **pseudo-labels** on the target with an **entropy**-based uncertainty filter, and (b) **aligns features** across systems with a distance-based loss so source and target sit in a shared representation space.

In plain words: train on a labelled source, carefully invent labels for confident target examples, and pull the two systems’ log features closer together so the detector generalises better.

Content (what they did and found)

They evaluate six transfer directions among three Loghub benchmarks: BGL, HDFS, and OpenStack (for example BGL → HDFS means BGL is source and HDFS is target). They compare against LogTAD and MetaLog using **F1-score** (a balance of precision and recall) and **MCC** (Matthews Correlation Coefficient, useful when classes are imbalanced).

From their Table 2, ELFA-Log’s **average F1** across the six scenarios is **71.23**, versus **64.18** for LogTAD and **66.53** for MetaLog. Average MCC is **0.452** for ELFA-Log, versus **0.312** and **0.401**. On the HDFS → BGL task, ELFA-Log reaches F1 **87.61** (LogTAD 70.65; MetaLog 80.04) and MCC **0.814** (LogTAD 0.417; MetaLog 0.514). Not every single cell is first place (for example MetaLog leads F1 on BGL → HDFS), but the paper reports better **average** performance and a higher median F1 with a tighter spread across tasks.

They also show that the best **pseudo-label confidence threshold** depends on the target domain (higher thresholds ~0.90–0.95 suit HDFS/BGL targets; OpenStack preferred a lower threshold ~0.8 because of stronger class imbalance).

How the paper is organised

1. **Introduction** — motivates scarce labels and cross-system transfer; states ELFA-Log’s contributions.

2. **Related work** — supervised, semi-supervised, and transfer log detectors (including LogTAD, MetaLog, PLELog).
3. **Preliminaries** — log parsing (Drain), templates, and the CSLAD problem across BGL/HDFS/OpenStack-style logs.
4. **Method** — ELFA-Log: pseudo-labelling with entropy filtering plus feature alignment.
5. **Experimental setup** — datasets, baselines, Precision/Recall/F1/MCC.
6. **Results** — main Table 2 comparison, robustness box plots, hyperparameter sensitivity.
7. **Threats to validity / limitations.**
8. **Conclusions** and future work.

Link to Kasi’s project / gap left

ELFA-Log is the natural **transfer comparator**: it is recent, peer-reviewed, open access, and designed for cross-system log anomaly detection. The gap it leaves for Kasi is clear. It still needs a **labelled source**, and it is tested on **classic Loghub corpora**, not on **AWS serverless Cloud-Watch** logs with **injected faults**. Kasi’s source-free setting asks what accuracy is forfeited when that labelled source is removed.

Article 2 — Ali et al. (2025): comprehensive ML study for log anomaly detection

Full citation: Ali, S., Boufaied, C., Bianculli, D., Branco, P. and Briand, L. (2025) ‘A comprehensive study of machine learning techniques for log-based anomaly detection’, *Empirical Software Engineering*, 30(5), 129. <https://doi.org/10.1007/s10664-025-10669-3>

Source of PDF used here: Author Accepted Manuscript from the University of Luxembourg ORBilu repository (<https://orbilu.uni.lu/bitstream/10993/57622/1/emse2025.pdf>, 68 pages). The Version of Record is the Springer open-access article at the DOI above. (A matching arXiv preprint also exists as arXiv:2307.16714; we used the ORBilu accepted manuscript because it is post-peer-review and cites the VoR DOI.)

Purpose (what problem it tackles)

The literature on log-based anomaly detection (LAD) heavily favours **deep learning**. Ali et al. argue that **traditional** machine learning (for example Random Forest and SVM) may still do well, and

that **semi-supervised** methods (which learn mainly from normal logs and need few or no anomaly labels) deserve the same careful comparison as supervised ones. They also say accuracy alone is not enough for engineers: **training time**, **prediction time**, and **sensitivity to hyperparameter tuning** matter in practice. Their goal is a large, systematic comparison on public labelled benchmarks.

Content (what they did and found)

They compare, on **seven** public labelled datasets (session-based: HDFS, Hadoop, F-dataset; message-based: Hades, BGL, Thunderbird, Spirit):

- **Supervised traditional:** SVM, Random Forest (RF)
- **Supervised deep:** LSTM, LogRobust, NeuralLog (two variants)
- **Semi-supervised traditional:** One-Class SVM (OC-SVM)
- **Semi-supervised deep:** DeepLog, Logs2Graphs

Four criteria: detection accuracy (mainly **F1-score**), time performance, and sensitivity of both accuracy and time to hyperparameters.

Key findings stated in the abstract and conclusion (and supported in the results sections):

- Supervised **traditional** and supervised **deep** methods fare **similarly** on detection accuracy and prediction time on most of these benchmarks (Kruskal–Wallis on supervised accuracy: $p = 0.88$, so no significant difference among those supervised techniques on that test).
- Supervised traditional methods are **less sensitive** to hyperparameter tuning than deep learning ones; among them, **Random Forest** shows the least sensitivity and is their practical recommendation.
- **Semi-supervised** techniques **yield significantly worse detection accuracy** than supervised techniques—even though they are attractive when anomaly labels are scarce. In other words, dropping labels has a real **accuracy cost** on these corpora.
- Example numbers from the text: on HDFS, DeepLog beats OC-SVM and Logs2Graphs by **26.07** and **20.14** percentage points of F1 respectively among semi-supervised methods; on session data, RF can reach F1 **96.58** on F-dataset while some deep models hit **0.00** there when anomalies are rare. On Hades (highly imbalanced), SVM reaches F1 **93.88** while several others drop (for example RF **72.73**).

So Ali et al. quantify both the **accuracy** picture and the **engineering cost** (time and tuning pain)—but always on the usual labelled public corpora.

How the paper is organised

1. **Introduction** — motivation and four evaluation criteria.
2. **Background** — LAD, parsing, traditional and deep methods.
3. **State of the art** — survey of prior empirical LAD studies (their large Table 1).
4. **Log representation** — how logs become features for the models.
5. **Empirical study design** — datasets, metrics, hyperparameters, statistical tests, research questions RQ1–RQ4.
6. **Results** — RQ1–RQ4 (supervised vs deep accuracy/time; semi-supervised accuracy/time and sensitivity), then discussion and threats.
7. **Conclusion and future work** — RF as a practical pick; call for diagnosis support beyond detection.

Link to Kasi’s project / gap left

Ali et al. strengthen the case that **labels matter**: semi-supervised (label-light) methods trail supervised ones on these benchmarks, and time/tuning costs are measurable. The gap for Kasi is that the study still uses **the same style of public labelled corpora**; it does **not** evaluate serverless Cloud-Watch telemetry, injected-fault campaigns, or a **source-free transfer** setup against a method like ELFA-Log.

Article 3 — Nguyen et al. (2025): silent failures in serverless systems

Full citation: Nguyen, C., Elmroth, E. and Bhuyan, M. (2025) ‘Silent failures in stateless systems: rethinking anomaly detection for serverless computing’, in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, pp. 8–19. <https://doi.org/10.1109/SOSE67019.2025.00006>

Source of PDF used here: Authors’ arXiv preprint **arXiv:2507.04969** (v3, 12 pages; title and authors verified on page 1). IEEE Xplore is typically paywalled; this arXiv copy is a legal open preprint from the Umeå University authors.

Availability note: Full text was obtained from arXiv (legal preprint). Content below uses that preprint. Where the proposal summarised the paper, we only keep points that also appear in the arXiv text.

Purpose (what problem it tackles)

Serverless platforms (AWS Lambda, Google Cloud Functions, Azure Functions, OpenWhisk, OpenFaaS, and so on) hide servers, scale per request, and bill by execution time. That makes apps easy to run—but hard to monitor. Functions are **short-lived**, **stateless**, and often **opaque** to the developer. Classic anomaly detectors assume long-running VMs or containers with stable CPU/memory streams. Nguyen et al. argue those assumptions break for serverless, so the field needs a rethink. They explicitly call this the **first comprehensive vision paper** on anomaly detection for serverless systems.

Content (what they did and found)

This is **not** a “we propose Detector X and beat baselines on F1” paper. It is a vision / agenda paper with an empirical **illustration**:

- They describe the four-layer serverless stack and warm vs cold starts.
- They map **operational** risks (cold-start amplification, noisy neighbours, orchestration delay, backend failures) and **adversarial** risks (DoS, Denial-of-Wallet, trigger abuse, evasion).
- In a case study on Apache OpenWhisk running an EfficientDet vision function, **warm** average execution time is about **280 ms** per request, while a **cold start** pushes end-to-end response time to about **10.5 seconds**—roughly a **37×** swing ($10.5 / 0.28 \approx 37.5$). They also show moderate (59 req/s for 3.9 s) and high-intensity (203 req/s for 4.9 s) burst DoS scenarios where latency and instance counts stay elevated after the burst.
- They argue that “normal” serverless elasticity looks a lot like anomalies, so simple thresholds fail.
- Their forward vision stresses **context-aware** detection and **multi-source data fusion**, plus real-time, lightweight, privacy-preserving, edge–cloud designs.
- On labelling, they state that acquiring accurate **ground-truth** anomaly labels in ephemeral serverless settings is **exceedingly difficult**, and they call for better data generation, weak labelling, and serverless-specific benchmarks. They do **not** ship and score a finished detector against labelled serverless log corpora.

How the paper is organised

1. **Section I — Introduction** — serverless adoption and why anomaly detection must be rethought.

2. **Section II — Background and motivation** — architecture, warm/cold starts, limits of traditional monitoring.
3. **Section III — Threat landscape** — operational vs adversarial threats, OpenWhisk case study figures.
4. **Section IV — Vision** — context-aware logic and multi-source fusion; design principles.
5. **Section V — Research opportunities** — labelling/benchmarks, explainability, observability models.
6. **Section VI — Realization challenges** — telemetry limits, deployment overhead, evolving threats.
7. **Section VII — Conclusion.**

Link to Kasi’s project / gap left

Nguyen et al. supply the **serverless problem framing** Kasi needs: ephemeral telemetry, cold-start noise, and labels that are hard to get. The gap they leave is intentional but important for the thesis: they **evaluate no production-style detector** with reported F1 on serverless logs, and they do not provide a transfer or source-free baseline comparison. Kasi’s artefact (CloudWatch logs + injected faults + ELFA-Log comparator) is exactly the kind of concrete evaluation this vision paper says the field still needs.

Closing: how the three papers form one gap

Taken together, the three studies each hold one piece of Kasi’s puzzle—but **no single study holds all four**:

Need for Kasi’s thesis	Zhao et al. (2025)	Ali et al. (2025)	Nguyen et al. (2025)
No labelled source (source-free)	No — needs labelled source	Semi-supervised is label-light, but not source-free transfer	Labels called exceedingly hard; no source-free detector eval
Serverless / CloudWatch-style telemetry	No — Loghub HDFS/BGL/OpenStack	No — seven classic log benchmarks	Yes — serverless focus
Injected-fault / usable ground truth for evaluation	Uses existing labelled corpora	Uses existing labelled corpora	Vision + OpenWhisk illustration; no full detector benchmark

Need for Kasi’s thesis	Zhao et al. (2025)	Ali et al. (2025)	Nguyen et al. (2025)
Transfer comparator (e.g. ELFA-Log-style)	Yes — ELFA-Log itself	Compares many ML families, not CSLAD transfer	No detector bake-off

So the project gap can be said in one sentence: **we still lack a measured answer for source-free log anomaly detection on serverless CloudWatch logs with injected-fault ground truth, compared against a modern transfer baseline such as ELFA-Log.** Zhao gives the comparator but not the setting; Ali shows the accuracy/time cost of going label-light but not on serverless; Nguyen gives the serverless setting but not the detector evaluation. Kasi’s thesis is designed to close that combined hole.

Sources checklist (what was downloaded)

1. **Zhao et al. (2025)** — full PDF: MDPI OA copy reused from `baseline_papers/Zhao_et_al_2025_ELFA-` (verified).
2. **Ali et al. (2025)** — full PDF: ORBilu Author Accepted Manuscript (Springer VoR DOI 10.1007/s10664-025-10669-3); not a pirate copy.
3. **Nguyen et al. (2025)** — full PDF: arXiv:2507.04969 (legal preprint of the IEEE SOSE 2025 paper).

End of plain-language explanation.